

IoTTrust 실기 과제 결과 보고서

1. 성능 개선

1.1 개선 내용

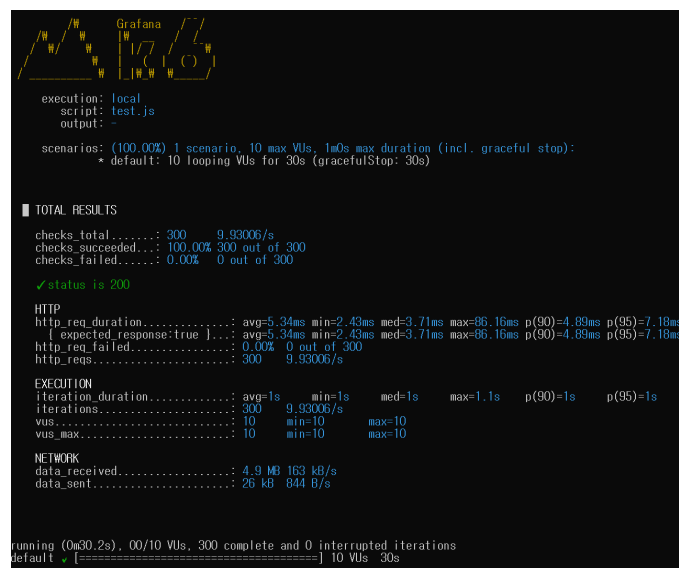
- **NestJS + Prisma 코드 리팩토링**
 - DTO 적용 및 **ValidationPipe** 전역 설정
 - **prisma.\$transaction** 사용으로 다중 주문 생성 시 트랜잭션 처리
- **Prisma 스키마 인덱스 추가**
 - **Order** 모델에 **@@index([userId])** 추가 → **userId** 기반 조회 성능 향상
- **대량 데이터 시드**
 - 사용자 50명, 사용자당 주문 200건(총 10,000건) 자동 생성

1.2 성능 테스트

- **k6 부하 테스트 결과**

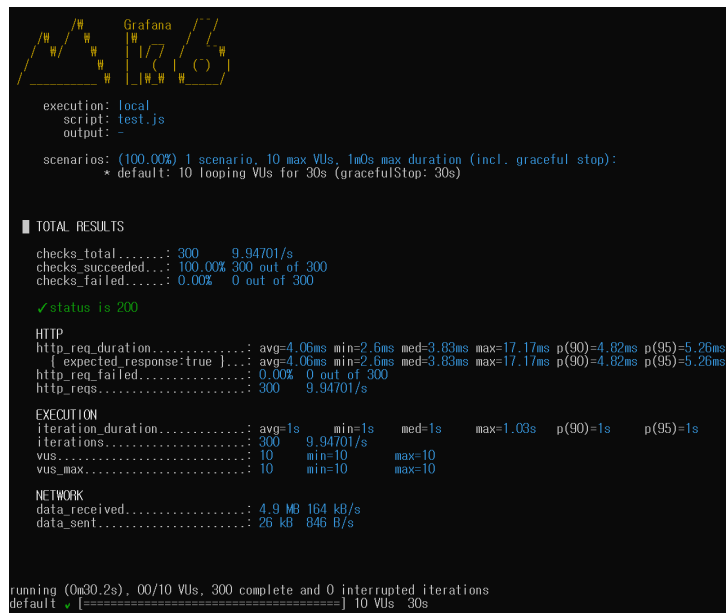
개선 전

- 평균 응답 시간: 5.34 ms
- 최대 응답 시간: 86.16 ms
- 최소 응답 시간: 2.43 ms
- p95 (95퍼센타일): 7.18 ms



개선 후

- 평균 응답 시간: 4.06 ms
- 최대 응답 시간: 17.17 ms
- 최소 응답 시간: 2.6 ms
- p95 (95퍼센타일): 4.82 ms



개선 효과

- 평균·최대·최소 모든 지표에서 성능이 향상. **최대 응답 시간은 약 80 % 감소**하여 가장 큰 개선 효과.

2. 모니터링 환경 구축

2.1 구성

- **Prometheus + Grafana**
 - Prometheus: 지표 수집
 - Grafana: 시각화 및 대시보드 구성
- **windows_exporter v0.31.3-amd64**
 - Windows OS 자원(CPU, 메모리, FD 등)을 Prometheus가 수집할 수 있도록 노드 익스포터 역할

2.2 설치 및 실행

windows_exporter

- 공식 릴리스 페이지에서 [windows_exporter-0.31.3-amd64.msi](#) 다운로드 및 설치
- 서비스로 등록 후 자동 실행되며 기본 포트 9182에서 메트릭 노출

Prometheus & Grafana (Docker)

```
# Prometheus
docker run -d --name prometheus -p 9090:9090 \
-v /your/path/prometheus.yml:/etc/prometheus/prometheus.yml prom/prometheus

# Grafana
docker run -d --name grafana -p 3000:3000 grafana/grafana
```

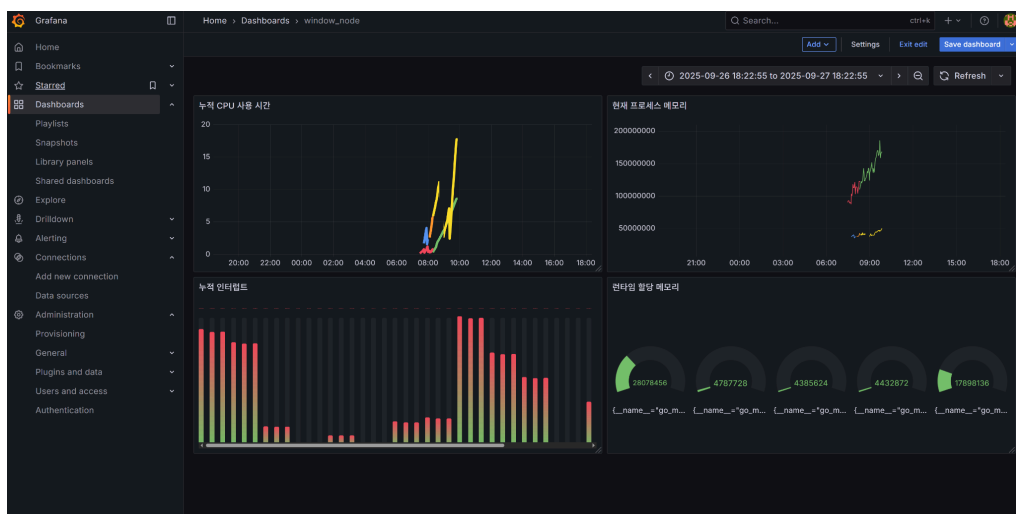
- Prometheus 설정 파일([prometheus.yml](#))의 [scrape_configs](#)에 [host.docker.internal:9182](#) (windows_exporter) 주소 추가
- Grafana 접속(<http://localhost:3000>) → Prometheus 데이터소스 등록 → 대시보드 생성

2.3 모니터링 대상 지표

- **process_cpu_seconds_total** : 프로세스가 사용한 누적 CPU 시간
- **windows_cpu_clock_interrupts_total** : 누적 클럭 인터럽트 수
- **process_resident_memory_bytes** : 현재 프로세스가 점유 중인 메모리
- **go_memstats_alloc_bytes** : Go 런타임이 힙에 할당한 메모리

2.4 시각화

- Grafana 대시보드에 위 4개 지표를 Time Series 또는 Gauge 패널로 구성



3. 결론

- 필수 요구사항인 성능 개선과 모니터링 환경 구축 완료
- 부하 테스트로 수정전의 소스와 수정후의 소스의 성능 차이를 비교
- Prometheus + Grafana + windows_exporter 기반 모니터링 환경을 구성하고, 핵심 지표(누적 CPU 사용, 인터럽트, 프로세스 메모리, Go 런타임 메모리)를 시각화 완료